

mess up, so use the `TAB` key. Second is that this will print a lot of data, fast. You can use `CTRL-C` to stop the command and stop all the output. Let's take a look at the `/turtle1/pose` topic.

```
$ ros2 topic echo /turtle1/pose
x: 5.4078755378723145
y: 7.081490516662598
theta: -1.0670461654663086
linear_velocity: 1.0
angular_velocity: 0.0
---
x: 5.4155988693237305
y: 7.067478179931641
theta: -1.0670461654663086
linear_velocity: 1.0
angular_velocity: 0.0
---
x: 5.423322677612305
y: 7.053465843200684
theta: -1.0670461654663086
linear_velocity: 1.0
angular_velocity: 0.0
---
<<GOING ON FOREVER>>
```

Let's examine what is going on. Between the dashes (`---`) is a single ROS message on our topic. If you examine the numbers closely you can see that they are changing, and doing so in relation to the movement of the turtle. Going back to our car example you can see how this would be useful for understanding the instantaneous velocity of each of our wheels.

Now that we have the basics down let's dig into a few of the optional arguments. We see a variety of commands that start with `--qos`. "QOS" here means "quality of service" and it is a really cool feature that is only in ROS 2. Without getting too technical, QOS is a way of asking for a certain level of networking robustness. A ROS system can operate over a network, and just like streaming video or video games, packets can get dropped or not get to their destination. The OS settings help you control which packets are the most important and should get the highest priority.

Most of the other commands deal with changing the output format of this CLI program, but there is one in particular that is super handy, and it is also new in ROS 2. The `--csv` flag stands for "comma separated values" and it is a very simple way of defining a spreadsheet. What this argument does is make the topic echo command output data in the comma separate value format. Many command lines allow you send data from the screen to a file, saving the data for later review or analysis. To do this file saving in Linux we use the `>` character followed by a file name. Below are two examples of using the `--csv` argument:

```
$ ros2 topic echo /turtle1/pose --csv
7.097168922424316,8.498645782470703,2.442624092102051,0.0,0.4000000059604645
7.097168922424316,8.498645782470703,2.449024200439453,0.0,0.4000000059604645
...
<<CTRL-C>>
$ ros2 topic echo /turtle1/pose --csv > mydata.csv
<<nothing happens>>
<<CTRL-C>>
```

The second command above creates a file called `mydata.csv`. You can look at it using a CLI utility called `less` (press `q` to quit), or open it with your favorite spreadsheet tool.

Now that we've looked at `ros2 topic echo` let's take a look at a few other topic sub commands. One thing you may have noticed is that topics can output a lot of data! More complex robots, like a self driving car, can saturate a high speed internet connection with how much data it produces. There are two topic sub commands that can be used to diagnose performance issues. The first sub command is `topic hz` which is the abbreviation of Hertz, the unit of frequency. The `hz` sub command will tell you how often a particular topic produces a message. Similarly there is the `topic bw` sub command, where `bw` stands for bandwidth, which is an engineering term related to the *volume* of data being produced. A high bandwidth connection can move more data, like high definition video, than a low bandwidth data, which might move a radio show. Let's take a look at the help for these two commands:

```
$ ros2 topic hz --help
usage: ros2 topic hz [-h] [--window WINDOW] [--filter EXPR] [--wall-time]
      topic_name
```

Print the average publishing rate to screen

positional arguments:

topic_name Name of the ROS topic to listen to (e.g. '/chatter')

optional arguments:

-h, --help show this help message and exit
 --window WINDOW, -w WINDOW window size, in # of messages, for calculating rate
 (default: 10000)
 --filter EXPR only measure messages matching the specified Python
 expression
 --wall-time calculates rate using wall time which can be helpful
 when clock is not published during simulation

```
$ ros2 topic bw --help
```

```
usage: ros2 topic bw [-h] [--window WINDOW] topic
```

Display bandwidth used by topic

positional arguments:

topic Topic name to monitor for bandwidth utilization

optional arguments:

-h, --help show this help message and exit
 --window WINDOW, -w WINDOW window size, in # of messages, for calculating rate
 (default: 100)

Both `bw` and `hz` follow the same pattern, they simply take in a topic name followed by a few optional arguments. The only argument worth noting is the `window` argument. Both of these commands calculate statistics for a series of messages; how many messages to use in calculating those statistics is the window size. The default value for `window` is 100, so when you call `ros2 topic bw` it will first collect 100 messages then use that data to calculate the average message size. Let's give it a shot (use `TAB` to complete and `CTRL-C` to exit).

```
$ ros2 topic hz /turtle1/pose
average rate: 60.021
   min: 0.001s max: 0.073s std dev: 0.00731s window: 65
average rate: 61.235
   min: 0.001s max: 0.073s std dev: 0.00523s window: 128
$ ros2 topic bw /turtle1/pose
Subscribed to [/turtle1/pose]
average: 1.44KB/s
   mean: 0.02KB/s min: 0.02KB/s max: 0.02KB/s window: 46
average: 1.52KB/s
   mean: 0.02KB/s min: 0.02KB/s max: 0.02KB/s window: 100
```

As we can see above, the `hz` command says that the topic is publishing messages at 60.021, where the unit is hz, or 60.021 times a second. Notice that the command gives the publishing frequency as an average, followed by the minimum, maximum, and standard deviation, in seconds. The bandwidth sub command is very similar; and we can see that the topic is producing 1.44 kilobytes of data per second. This command has similar outputs around the minimum, maximum, and mean.

One tool that is handy when exploring topics is understanding their type. While we have already looked at the `interface` command to see what integral types make up a topic, the `topic` command has both a tool to query the type of a topic, and a means to search all topics for a specific type. If all you want to know is a topic's type you can use the `type` command which will return a type that can then be further explored with the `interface` command. If instead you would like to know what topics use a particular message type you can use the `topic find` command, sub command pair. Both the `topic type` and `topic interface` command, sub command pairs have a very limited set of optional arguments, so we simply provide them with our desired topic or message type. Let's take a look at these two commands together:

```
$ ros2 topic type --help
usage: ros2 topic type [-h] topic_name
Print a topic's type
```

```
positional arguments:
  topic_name  Name of the ROS topic to get type (e.g. '/chatter')
```

```
optional arguments:
  -h, --help  show this help message and exit
$ ros2 topic type /turtle1/pose
turtlesim/msg/Pose
$ ros2 topic find --help
usage: ros2 topic find [-h] [-c] [--include-hidden-topics] topic_type
```

Output a list of available topics of a given type

```
positional arguments:
  topic_type  Name of the ROS topic type to filter for (e.g.
              'std_msgs/msg/String')
```

```
optional arguments:
  -h, --help  show this help message and exit
  -c, --count-topics  Only display the number of topics discovered
  --include-hidden-topics  Consider hidden topics as well
$ ros2 topic find turtlesim/msg/Pose
/turtle1/pose
```

The last sub command for the topic command is `pub`, which simply means publish. It allows you to publish a command to any ROS topic from the command line. While you shouldn't need

to use this command regularly it can be particularly handy for testing and debugging when you are building a robot system. The `pub` command has a number of optional arguments that allow you to send one or more messages, and with different quality of service (QoS) presets. The format of the command is `ros2 topic pub TOPIC_NAME MESSAGE_TYPE VALUES`, which means for it to work successfully you must include a target topic, the topic's message type, and finally the message's values. The values for the message are specified in the YAML format and we can use the `interface show` command to understand the format. To illustrate the utility of this command we'll issue a message to rotate and stop our turtle by publishing to the `/turtle1/cmd_vel/` topic. Let's first take a look at the `topic pub` documentation before we construct our command:

```
$ ros2 topic pub --help
usage: ros2 topic pub [-h] [-r N] [-p N] [-1] [-n NODE_NAME]
                        [--qos-profile
                        {system_default,sensor_data,services_default,parameters,parameter_events,action_status_default}]
                        [--qos-reliability {system_default,reliable,best_effort}]
                        [--qos-durability {system_default,transient_local,volatile}]
                        topic_name message_type [values]
```

Publish a message to a topic

positional arguments:

topic_name	Name of the ROS topic to publish to (e.g. '/chatter')
message_type	Type of the ROS message (e.g. 'std_msgs/String')
values	Values to fill the message with in YAML format (e.g. "data: Hello World"), otherwise the message will be published with default values

optional arguments:

-h, --help	show this help message and exit
-r N, --rate N	Publishing rate in Hz (default: 1)
-p N, --print N	Only print every N-th published message (default: 1)
-1, --once	Publish one message and exit
-n NODE_NAME, --node-name NODE_NAME	Name of the created publishing node
--qos-profile {system_default,sensor_data,services_default,parameters,parameter_events,action_status_default}	Quality of service preset profile to publish with (default: system_default)
--qos-reliability {system_default,reliable,best_effort}	Quality of service reliability setting to publish with (overrides reliability value of --qos-profile option, default: system_default)
--qos-durability {system_default,transient_local,volatile}	Quality of service durability setting to publish with (overrides durability value of --qos-profile option, default: system_default)

Since we want to manually move our turtle we will use the `--once` flag to issue our command once. It is worth noting that the message type used to command the velocity of the turtle is complex in that it is made up of other message types so we'll have to query the base message type. Here's a rough summary of what we will do:

- Print the `cmd_vel` topic type using `ros2 topic type`, which is `geometry_msgs/msg/Twist`
- Determine the structure of the `Twist` message type using `interface show`.
- Determine the structure of the `Vector3`, which is part of the `Twist` message type using `interface show` command a second time.

- Create the YAML syntax for our command. Note the YAML syntax below as it is rather tricky! The YAML is wrapped in single quotes and a top level set of curly braces, while subsequent levels follow the pattern of `name:value`, and `name:`
`{name1:val1,name2:val2}` for nested types like the `Twist` command.
- Issue the command using `ros2 pub`.

```
$ ros2 topic type /turtle1/cmd_vel
geometry_msgs/msg/Twist
$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3  linear
Vector3  angular
$ ros2 interface show geometry_msgs/msg/Vector3
# This represents a vector in free space.

float64 x
float64 y
float64 z

$ ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist '{linear: {x:
4.0,y: 4.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}'
```

If you did everything correctly you should have moved around the turtle on the screen. Try changing the command to draw a small picture.

ROS 2 Services and Actions

As discussed previously, services are the name given to short, synchronous robot behaviors that can be done quickly, like turning on lights and switching components on or off. Actions are longer term, asynchronous, tasks that may have intermediate steps. A classic example of an action is navigation: a robot is provided a goal position and asked to navigate to that goal. Try as the robot might, since it cannot move infinitely fast, it takes time to move to a goal and sometimes its path may become blocked.

These two primitives are the backbone of most robotic systems using ROS, and learning how to use them via the command line will allow you quickly and easily command a robot to complete a task for you. To aid in clarity of this section we'll also touch on the `ros2 node` command to determine what node, or software process, is conducting a particular action or service.

Let's get nodes out of the way quickly. ROS nodes are small programs, running in their own process. A ROS system can have ten, hundreds, or even thousands of nodes running concurrently. Moreover, a ROS system can have multiple copies of the same node running

concurrently on the same system. In the case of our turtle simulation we can actually create multiple turtles, each with their own node, all running the exact same program. ROS Nodes, like ROS topics, have namespaces so that you can address specific nodes in the case where multiple copies of the same node (program) are running. Let's dig in a bit by restarting our turtle simulation in a terminal using `ros2 run turtlesim turtlesim_node`. Now in a new terminal let's first examine what `ros2 node` has to offer by asking for help.

```
$ ros2 node --help
usage: ros2 node [-h]
           Call `ros2 node <command> -h` for more detailed usage. ...
```

Various node related sub-commands

optional arguments:

`-h, --help` show this help message and exit

Commands:

`info` Output information about a node
`list` Output a list of available nodes

Call ``ros2 node <command> -h`` for more detailed usage.

Much like topics, we see two sub commands, `info` and `list`. `node list` works much the same as `topic list` and simply prints a list of all running nodes. Let's see what is running on our system:

```
$ ros2 node list
/turtlesim
```

We have a single node running called "turtlesim". `node info` works in a way very similar to `topic info` except that it lists information about the nodes we give it. Let's call it with our single ROS Node `/turtlesim` as its argument:


```

$ ros2 node info /turtlesim
/turtlesim
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /turtle1/color_sensor: turtlesim/msg/Color
  /turtle1/pose: turtlesim/msg/Pose
Service Servers:
  /clear: std_srvs/srv/Empty
  /kill: turtlesim/srv/Kill
  /reset: std_srvs/srv/Empty
  /spawn: turtlesim/srv/Spawn
  /turtle1/set_pen: turtlesim/srv/SetPen
  /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
  /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
  /turtlesim/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /turtlesim/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /turtlesim/get_parameters: rcl_interfaces/srv/GetParameters
  /turtlesim/list_parameters: rcl_interfaces/srv/ListParameters
  /turtlesim/set_parameters: rcl_interfaces/srv/SetParameters
  /turtlesim/set_parameters_atomically:
rcl_interfaces/srv/SetParametersAtomically
Service Clients:

Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
Action Clients:

```

Wow, that's a lot of information, some of which looks familiar. We can see all the topics that the node subscribes to, as well as all the nodes it publishes to. We can also see a number of "action servers" and "service servers". It is worth noting the client and server relationship here. Since ROS may have multiple nodes running some nodes may offer services (these are servers), and other ROS nodes may call those servers (these are the clients). The clients can be other ROS nodes, or for these examples, a human using the CLI.

The command line interface for services and actions are very similar, in fact they both have only four sub commands. Let's run the `action` and `service` commands and compare them:

```
$ ros2 action --help
usage: ros2 action [-h]
                Call `ros2 action <command> -h` for more detailed usage.
...
```

Various action related sub-commands

optional arguments:

-h, --help show this help message and exit

Commands:

info	Print information about an action
list	Output a list of action names
send_goal	Send an action goal
show	Output the action definition

Call `ros2 action <command> -h` for more detailed usage.

```
$ ros2 service --help
usage: ros2 service [-h] [--include-hidden-services]
                Call `ros2 service <command> -h` for more detailed usage.
...
```

Various service related sub-commands

optional arguments:

-h, --help	show this help message and exit
--include-hidden-services	Consider hidden services as well

Commands:

call	Call a service
find	Output a list of available services of a given type
list	Output a list of available services
type	Output a service's type

Call `ros2 service <command> -h` for more detailed usage.

We can see that both commands have a `list` command that gives a list of available services or actions. If we had multiple nodes running and wanted to see every service offered, calling `ros2 node info` on each node would be very inefficient, particularly if we had tens, or even hundreds of nodes running. In this case it would be much more efficient to use the `list` commands for the action and service commands. We can run these commands below and see that we get roughly the same list of actions and services listed in our single nodes: